



SOFTWARE DEVELOPMENT

CO2401



G21328023

K.A. IDUSHA PIUMIKA

27/03/2026

CO2401 Software Development Assignment

Test Driven Development of a Smart Traffic Controller Project: A Reflection Report

1. Introduction

The Smart Traffic Controller project used a Test Driven Development (TDD) method in C# to create an intelligent traffic control system that can control vehicle signal lights, pedestrian signal lights and timing using the object-oriented approach of programming.

The Traffic Controller created in this project was designed to work with various external dependencies such as the vehicle signal manager, pedestrian signal manager, time manager, web service and email service. Each of these dependencies were abstracted using interfaces, and then were tested using mock objects.

The following tools were used in the development of this project:

- C# (.NET)
- Visual Studio
- NUnit4 for Unit Testing (testing)
- NSubstitute for Mocking Dependencies (mock object creation)

The testing process used during the course of developing this project adhered to a structured TDD workflow to ensure that the Smart Traffic Controller met the specifications of the client and the completed system was reliable and correct.

2. What was the project about?

For The TrafficController Class Project, I was required to create an incremental implementation of the TrafficController Class per TDD based on Level 1 and Level 2 specifications.

I started with the development of the Level 1 unit tests like:

- Constructor initialization,
- Intersection ID to lowercase conversion
- Obtaining getter methods.
- Directly setting state through SetStateDirect().

Some of the tests were failing due to the inability to convert input to lower case or not validating whether or not the state being set was a valid state. I was able to fix these by implementing functionality to handle input normalization through the .ToLower() function along with input state validation against a pre-defined list of valid states.

For Level 2, I implemented:

- State transition logic in the SetCurrentState() method,
- Validation of initial state and throwing exceptions when invalid state is passed into constructor,
- Interface-based dependency injection and
- A method called GetStatusReport() to provide an aggregate status of several different services.

I tested with the help of NSubstitute to create a mock for the IVehicleSignalManager, IPedestrianSignalManager, and ITimeManager. By doing this, I was able to produce expected results different from those provided by an actual implementation.

In order to verify the output of the GetStatusReport() method by combining outputs from several dependencies, I created mock objects that returned pre-determined status strings, such as, “**VehicleSignal,OK**” etc.

When I wrote the test cases, I encountered a problem ensuring that invalid state transitions would not alter the internal state of the mock object. I resolved this by adding conditional checks prior to assigning any new value.

Another challenge was determining how best to structure my test cases. To overcome this, I used the Arrange-Act-Assert pattern as follows:

1. Arrange — create objects and mocks for the test case
2. Act — call the method under test
3. Assert — confirm that the expected result was produced

In general, the actual results met my expectations, once the implementation of the system and the test cases were aligned.

3. So What?

The assignment allowed me to better understand how Test Driven Development (TDD) and Object Oriented Programming (OOP) can be applied.

The process of writing tests before doing the implementation will change the way I work through problems. The test case would define how I expected the code to react — so I concentrated more on defining what that expected result was rather than writing the actual code first. This created a much more organized method for writing code, therefore minimizing the chance of introducing a bug.

For example, when implementing the state transitions, I created test cases for both valid and invalid transitions before I began writing my code. This provided some direction to my implementation process and also ensured that I accepted only valid state transitions.

Another valuable experience was using mocking frameworks (like NSubstitute). Utilizing mock objects helped isolate the TrafficController class from its dependent classes; therefore, when running unit tests, the test only focused on logic inside the TrafficController class and not the interaction with external systems.

One of my strengths completed during this assignment was that I adhered closely to the TDD cycle and organized test cases in a clear manner. I also utilized parameterized tests, which will reduce redundancy and improve readability.

Although I had some strengths there was still a need to improve my understanding of some of the advanced TDD scenarios, especially related to more complex interactions between multiple dependency injections and/or time based logic.

Using AI tools through the course of development helped me gain a better understanding of how to organize classes, create unit tests, and implement dependency injection.

4. Now What?

I'll keep using TDD in future projects as it:

- Recommends superior designs for software.
- Helps make the code more reliable.
- Allows the identification of defects early on in the development process.

If I had to do a similar type of project again, I would:

1. Devote additional time to creating test cases prior to building the code.
2. Enhance testing for edge cases.
3. Provide clearer naming of tests and improved maintainability.
4. Study more advanced mocking methodologies.

Areas in which I am looking to improve include:

- Advanced unit testing techniques.
- Better methods for dealing with complex state machines.
- A better understanding of dependency injection patterns.
- Improved methods for debugging and refactoring code.

5. Conclusions

This project was a great opportunity for me to increase my knowledge of developing object-oriented software and applying structured software development methodologies, such as TDD.

Through this project, I learned how to:

1. Develop a system with the use of interfaces and abstraction.
2. Build instances of a class with regard paying attention to many different requirements of a project.
3. Develop metrics to measure the quality of the software I developed via unit testing.
4. Utilise NUnit and NSubstitute effectively.

The TDD methodology provided me with the means to produce a system in a controlled and testable manner. By writing tests prior to developing the system, I was able to develop a better system design and to continue to have high confidence in the correctness of the software I developed as I created additional functionality.

Overall, this project allowed me to have an opportunity to practice my knowledge of software engineering methodologies in a real-world situation and allowed me to renew my confidence in developing high-quality, maintainable software.

Appendices

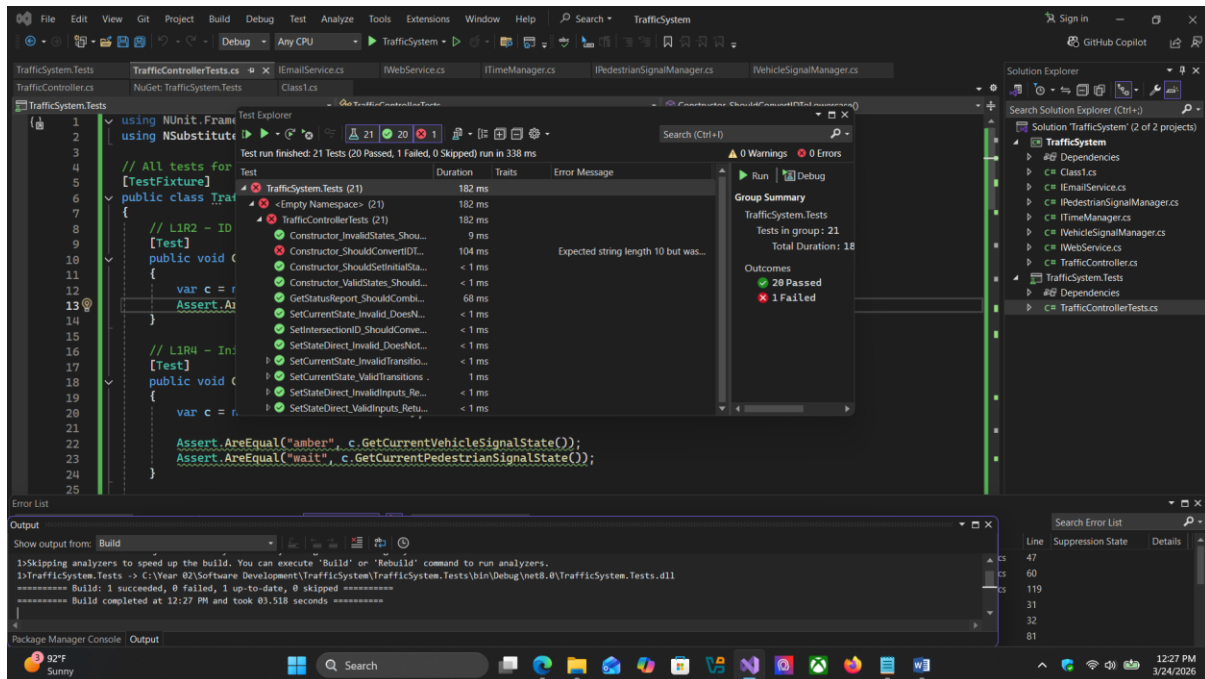
a. Requirements checklist

Level One	Level Two	Level Three
L1R1☒	L2R1☒	L3R1☐
L1R2☒	L2R2☒	L3R2☐
L1R3☒	L2R3☒	L3R3☐
L1R4☒	L2R4☒	L3R4☐
L1R5☒		L3R5☐

b. Appendix b – Evidence of Development and Testing

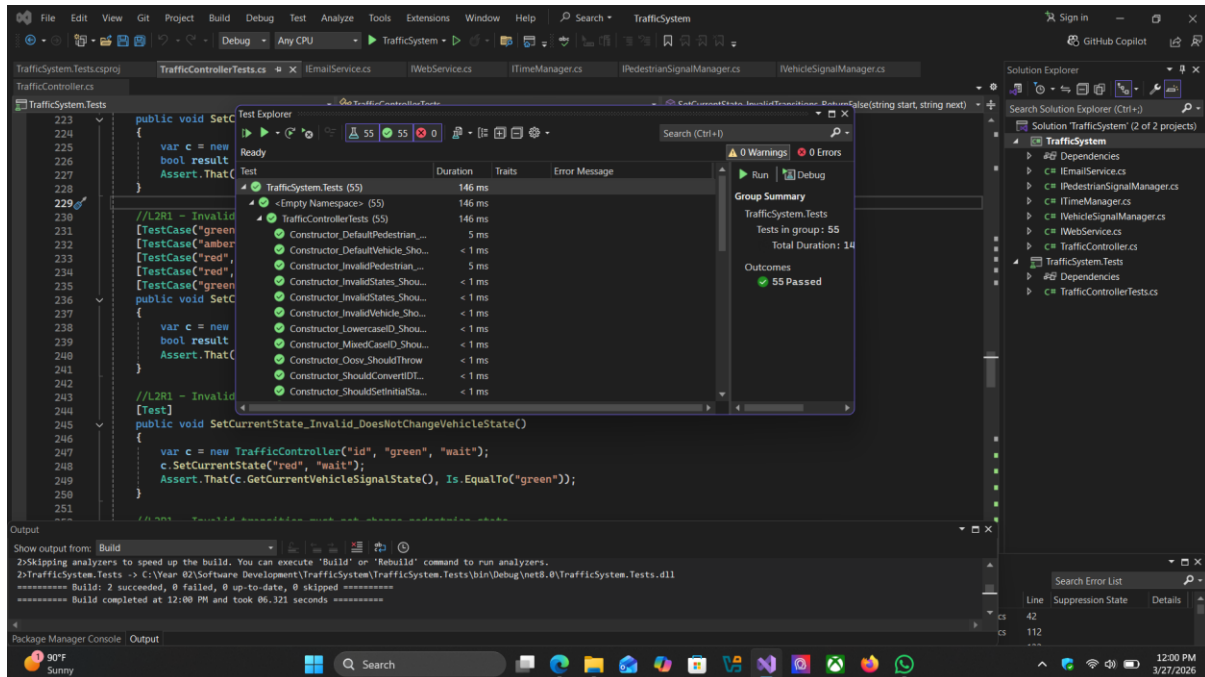
1. Test Failure and Fix

Subheading: Test Failure and Correction (TDD Cycle)



This screenshot shows a unit test failing in the NUnit Test Explorer due to an incorrect expected value. This demonstrates the “Red” phase of the Test Driven Development (TDD) cycle, where a test is written and executed before the implementation is corrected.

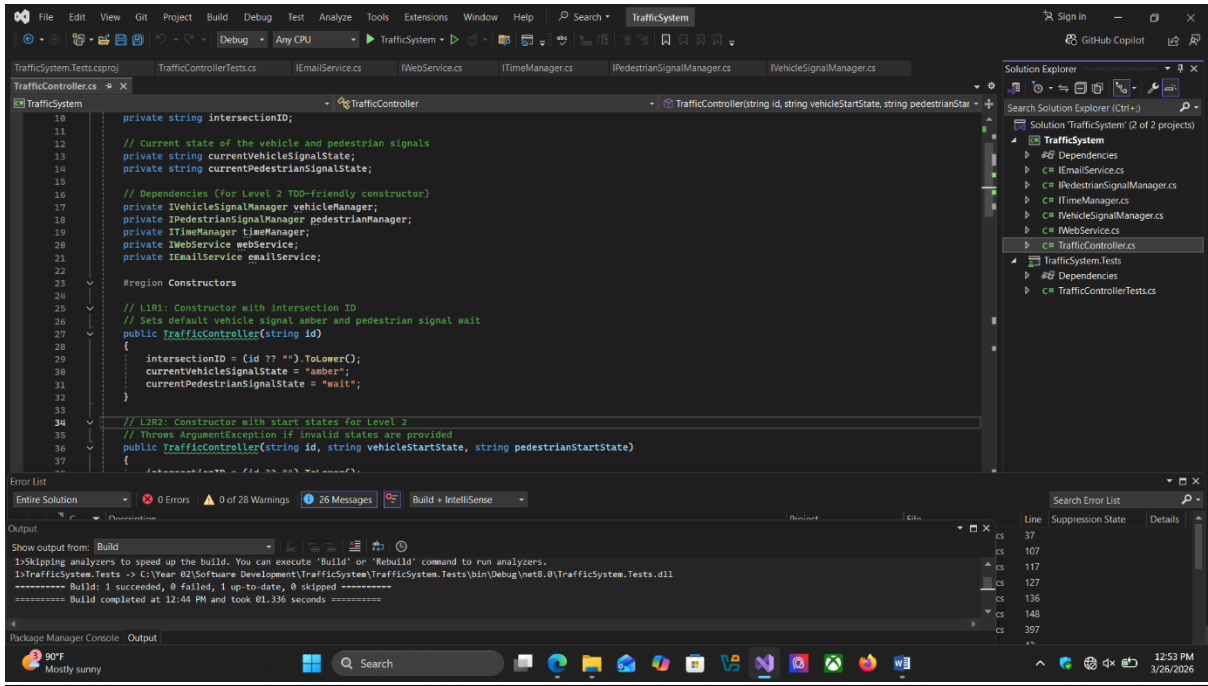
Test After Correction



This screenshot shows the same test passing after correcting the expected value. This represents the “Green” phase of TDD, where the implementation is updated to satisfy the test conditions.

2. Code Implementation

Subheading: TrafficController Implementation and Unit Tests

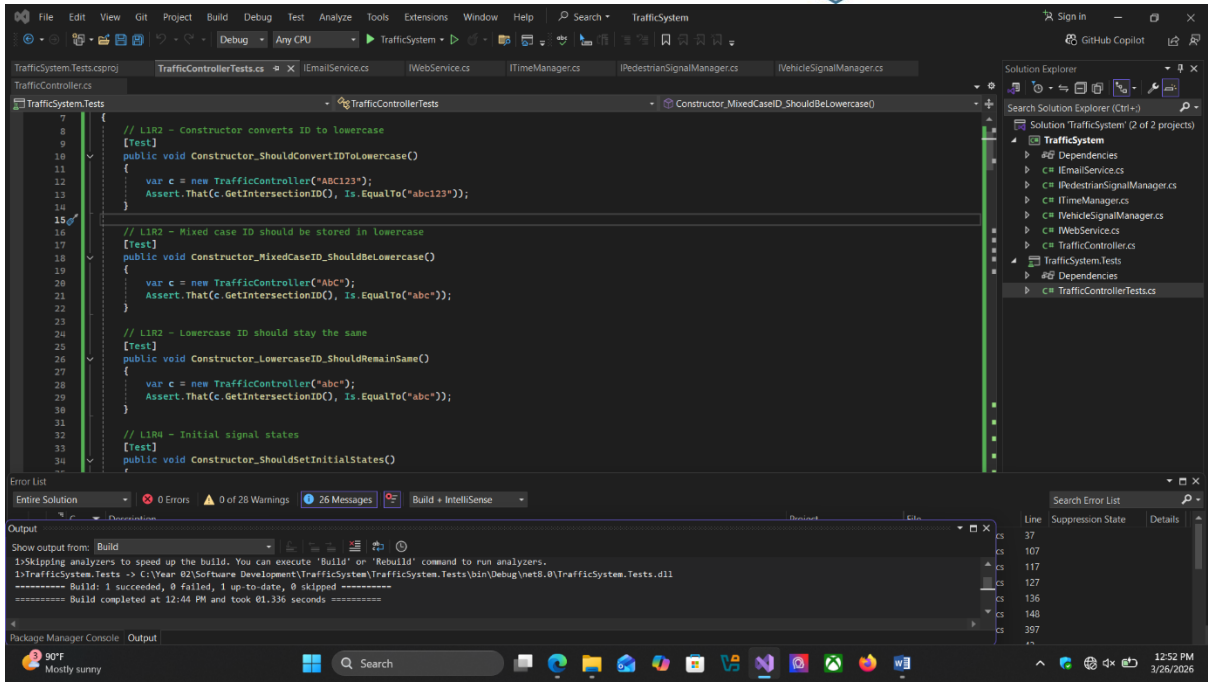


The screenshot displays the Visual Studio IDE with the `TrafficController.cs` file open. The code implements the `TrafficController` class, which manages the state of a traffic intersection. It includes private fields for `IntersectionID`, `currentVehicleSignalState`, and `currentPedestrianSignalState`. The class has two constructors: `L1R1` (Level 1) and `L2R2` (Level 2). The `L1R1` constructor initializes the intersection ID and sets default signal states. The `L2R2` constructor initializes the intersection ID and sets default signal states based on the provided start states. The class also includes dependencies for `IVehicleSignalManager`, `IPedestrianSignalManager`, `ITimeManager`, `IWebService`, and `IEmailService`.

The build output shows the following messages:

```
1>Skipping analyzers to speed up the build. You can execute 'Build' or 'Rebuild' command to run analyzers.
1>TrafficSystem.Tests -> C:\Year 02\Software Development\TrafficSystem\TrafficSystem.Tests\bin\Debug\net8.0\TrafficSystem.Tests.dll
----- Build: 1 succeeded, 0 failed, 1 up-to-date, 0 skipped -----
----- Build completed at 12:44 PM and took 01.336 seconds -----
```

This screenshot shows the implementation of the `TrafficController` class, including Level 1 and Level 2 requirements such as state management, input validation, and controlled state transitions using object-oriented principles.



```

7  {
8  // L1R2 - Constructor converts ID to lowercase
9  [test]
10 public void Constructor_ShouldConvertIDToLowercase()
11 {
12     var c = new TrafficController("ABC123");
13     Assert.That(c.GetIntersectionID(), Is.EqualTo("abc123"));
14 }
15
16 // L1R2 - Mixed case ID should be stored in lowercase
17 [test]
18 public void Constructor_MixedCaseID_ShouldBeLowercase()
19 {
20     var c = new TrafficController("Abc");
21     Assert.That(c.GetIntersectionID(), Is.EqualTo("abc"));
22 }
23
24 // L1R4 - Lowercase ID should stay the same
25 [test]
26 public void Constructor_LowercaseID_ShouldRemainSame()
27 {
28     var c = new TrafficController("abc");
29     Assert.That(c.GetIntersectionID(), Is.EqualTo("abc"));
30 }
31
32 // L1R4 - Initial signal states
33 [test]
34 public void Constructor_ShouldSetInitialStates()
35 {
36 }
37 }

```

Output

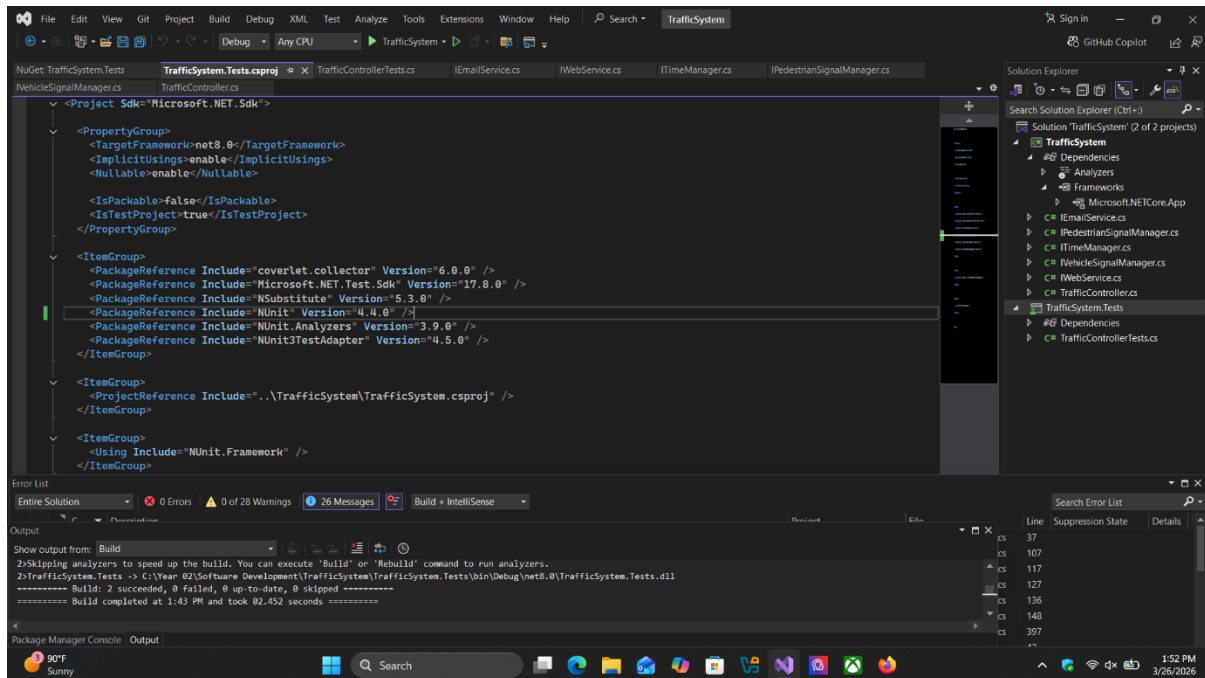
```

1>Skipping analyzers to speed up the build. You can execute 'Build' or 'Rebuild' command to run analyzers.
1>TrafficSystem.Tests -> C:\Vlan\02Software Development\TrafficSystem\TrafficSystem.Tests\bin\Debug\net8.0\TrafficSystem.Tests.dll
===== Build: 1 succeeded, 0 failed, 1 up-to-date, 0 skipped =====
===== Build completed at 12:44 PM and took 01.336 seconds =====

```

The above screenshot shows the unit test class developed using the NUnit framework. The tests follow the Arrange-Act-Assert pattern and include parameterised test cases to validate both valid and invalid scenarios.

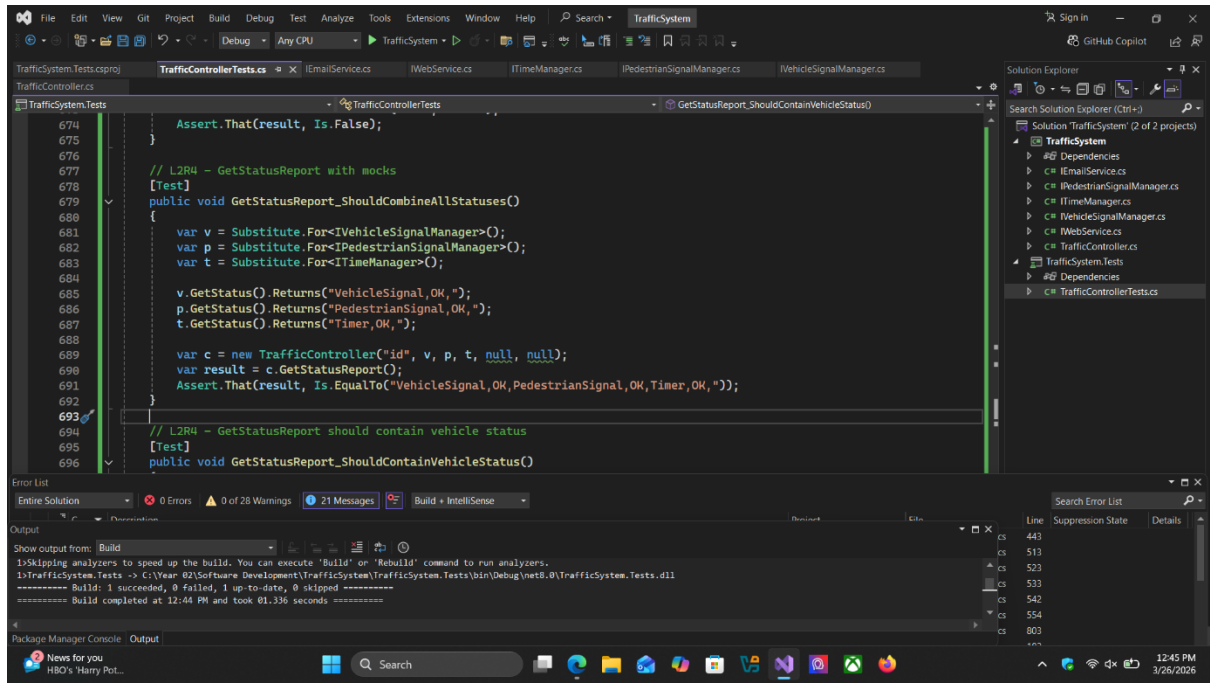
3. Testing Framework Configuration



This figure shows the configuration of NuGet packages used in the test project, including NUnit (version 4.4.0) for unit testing, NSubstitute for mocking dependencies, and Microsoft.NET.Test.Sdk for running tests. This setup ensures that the testing environment is correctly configured for implementing and executing unit tests.

4. Mocking (NSubstitute)

Subheading: Dependency Injection and Mocking



```

674     Assert.That(result, Is.False);
675 }
676
677 // L2R4 - GetStatusReport with mocks
678 [Test]
679 public void GetStatusReport_ShouldCombineAllStatuses()
680 {
681     var v = Substitute.For<IVehicleSignalManager>();
682     var p = Substitute.For<IPedestrianSignalManager>();
683     var t = Substitute.For<ITimeManager>();
684
685     v.GetStatus().Returns("VehicleSignal,OK,");
686     p.GetStatus().Returns("PedestrianSignal,OK,");
687     t.GetStatus().Returns("Timer,OK,");
688
689     var c = new TrafficController("id", v, p, t, null, null);
690     var result = c.GetStatusReport();
691     Assert.That(result, Is.EqualTo("VehicleSignal,OK,PedestrianSignal,OK,Timer,OK,"));
692 }
693
694 // L2R4 - GetStatusReport should contain vehicle status
695 [Test]
696 public void GetStatusReport_ShouldContainVehicleStatus()

```

Output

```

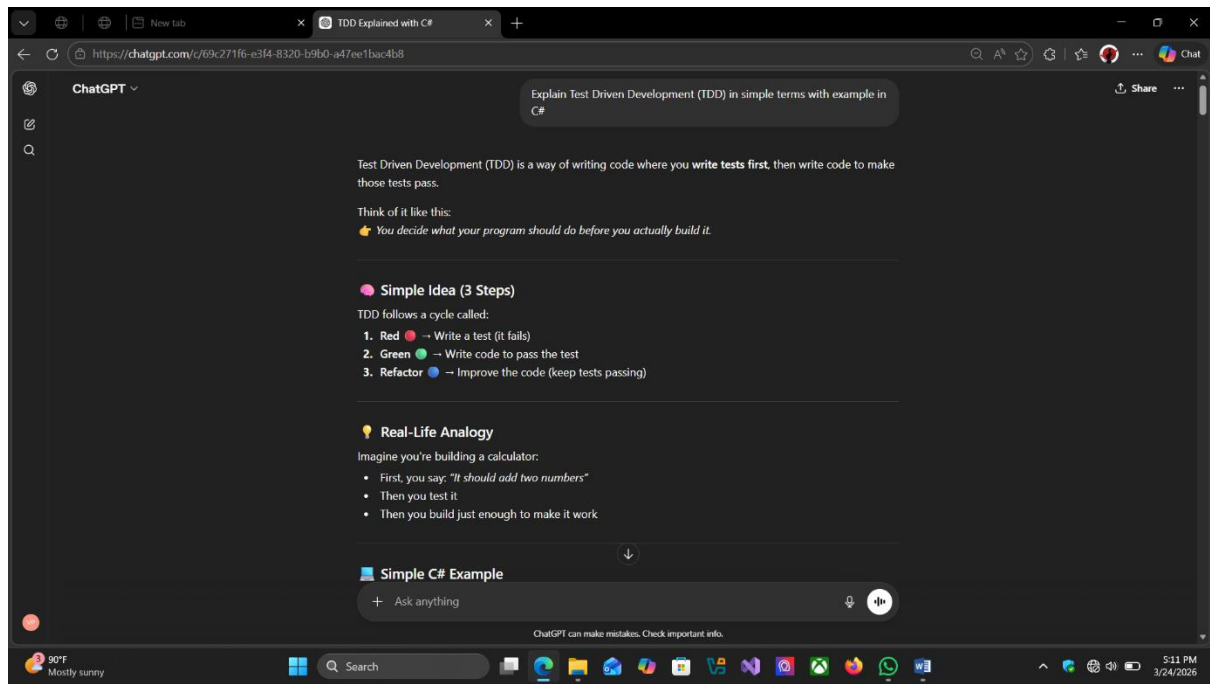
1>Skipping analyzers to speed up the build. You can execute 'Build' or 'Rebuild' command to run analyzers.
1>TrafficSystem.Tests -> C:\Year 02\Software Development\TrafficSystem\TrafficSystem.Tests\bin\Debug\net8.0\TrafficSystem.Tests.dll
***** Build: 1 succeeded, 0 failed, 1 up-to-date, 0 skipped *****
***** Build completed at 12:44 PM and took 01.336 seconds *****

```

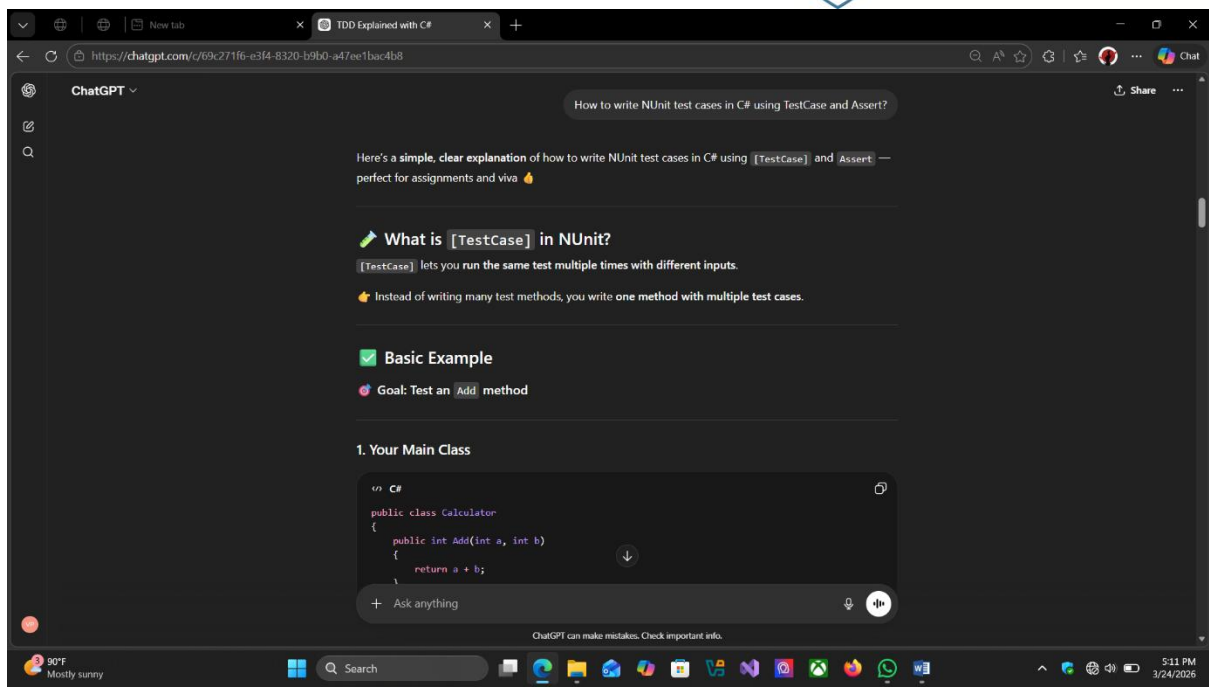
This figure demonstrates the use of the NSubstitute framework to create mock objects for external dependencies. The mocked methods are configured using the `.Returns()` function to simulate expected outputs, allowing isolated unit testing of the `TrafficController` class

5. GenAI Usage

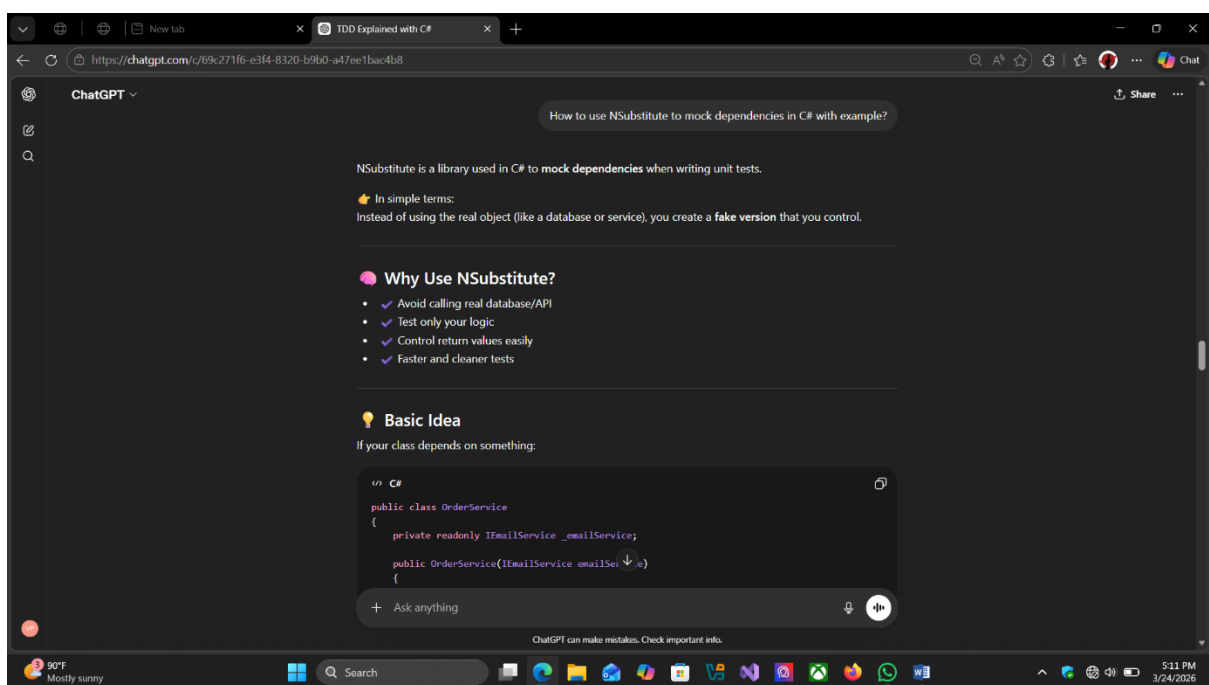
Use of Generative AI for Learning and Development.



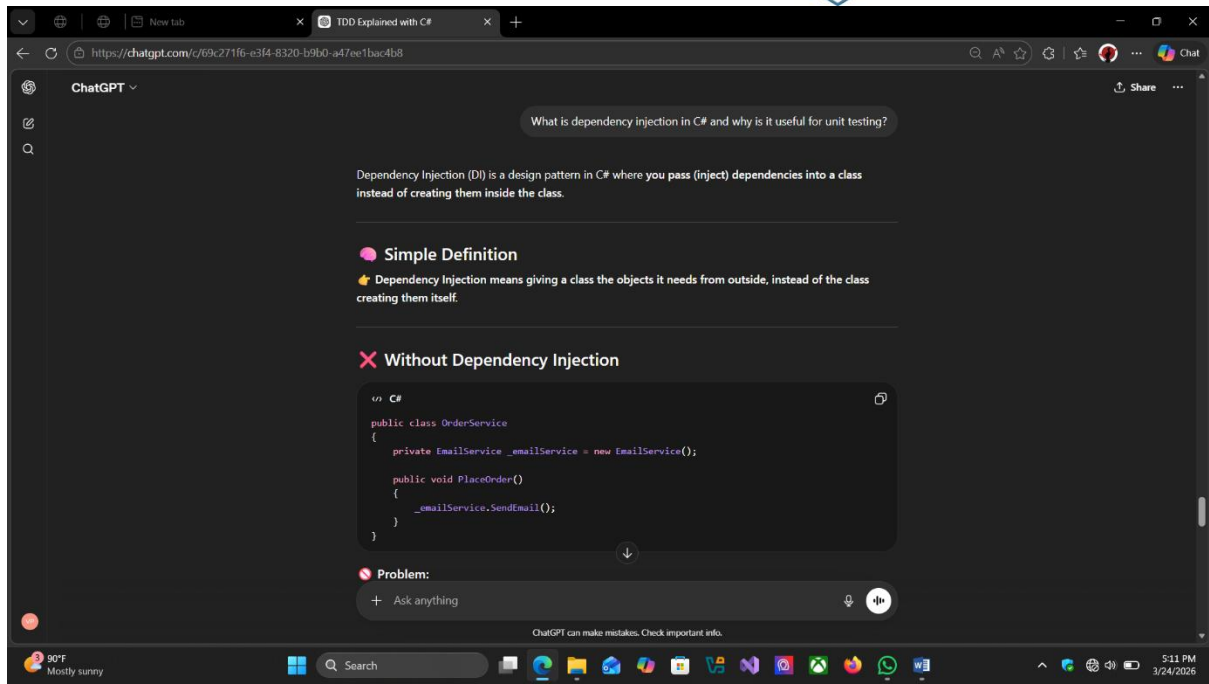
This figure shows a prompt requesting an explanation of Test Driven Development (TDD), which helped in understanding the Red-Green-Refactor cycle used throughout the assignment.



This figure shows a prompt requesting guidance on writing NUnit test cases using TestCase and Assert methods, which supported the development of structured and reusable unit tests.



This figure shows a prompt related to using NSubstitute for mocking dependencies, which was applied in testing the TrafficController class without relying on real implementations.



This figure shows a prompt explaining dependency injection in C#, which helped in designing the `TrafficController` class to be more testable and modular.

References

- Jurgen M. & Simo M. (2014), *Effects of Test-Driven Development: A Comparative Analysis of Empirical Studies*. Available at: https://www.researchgate.net/publication/256848134_Effects_of_Test-Driven_Development_A_Comparative_Analysis_of_Empirical_Studies
- A Golmohammadi, Man Z. & Andrea A. (September 2023) *.NET/C# instrumentation for search-based software testing*. *Software Quality Journal*. Available at: https://www.researchgate.net/publication/373599686_NETC_instrumentation_for_search-based_software_testing
- Afroz (22 Feb. 2025) *Mastering NUnit Testing: A Practical Guide to Better Unit Tests*. Available at: <https://medium.com/@afrozmoht/mastering-nunit-testing-a-practical-guide-to-better-unit-tests-9fa7654e0b7c>
- Folorunsho H. (21 Feb. 2024), *Understanding Mocks in Unit Testing*. Available at: <https://www.newline.co/@akinwalehabib/understanding-mocks-in-unit-testing--54f7d5e4>
- Kamlesh S. (05 Jan. 2024), *Dependency injection in c# with an example*. Available at: <https://medium.com/@kamlesh90/dependency-injection-in-c-with-an-example-8b067bfb0aa>
- Vidushan T. & Budditha H. (July 2022) *A STUDY ON OBJECT-ORIENTED DESIGN PRINCIPLES AND PATTERNS*. Available at: https://www.researchgate.net/publication/365683164_A_STUDY_ON_OBJECT-ORIENTED_DESIGN_PRINCIPLES_AND_PATTERNS
- S. Izzat & Nada N. Saleem. (June 2023), *Software Testing Techniques and Tools: A Review*. Available at: https://www.researchgate.net/publication/371214561_Software_Testing_Techniques_and_Tools_A_Review

- James B. (July 2023), *Exploring the Role of Test-Driven Development in Agile Software Development*. Available at: https://www.researchgate.net/publication/386135621_Exploring_the_Role_of_Test-Driven_Development_in_Agile_Software_Development